# TD No5: Gestion de la mémoire

October 20, 2026

## 1 Mémoire d'un processus

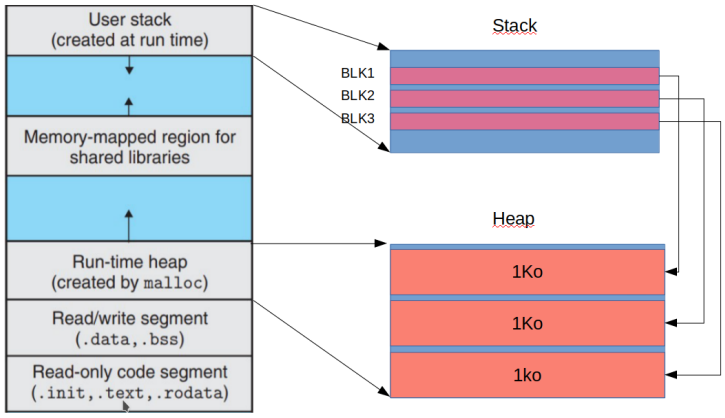
### 1.1 Segments

**Question1** Compilez et exécutez le programme **E1_addr_mapping.c** et commenter le résultat.

**Question2** Quel est le lien avec ce que vous connaissez de la mémoire virtuelle et de la gestion des pages ?

### 1.2 Stack et malloc

**Question1** Tapez un programme qui conduit à état suivant:




**Question2** Affichez les adresses des pointeurs et de leur contenu et vérifiez si le sommet de la pile et du tas évoluent-ils dans le sens prévu ?

### 1.3 Stack frame

Créez deux fonctions f1, f2 :

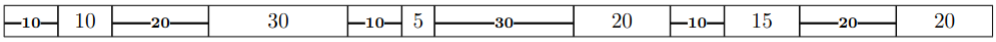
1. f2 crée 3 blocs et affiche l'adresse des variables BLK et l'adresse des blocs de données alloués.
2. f1 crée 3 blocs et affiche l'adresse des variables BLK et l'adresse des bloc de données alloués puis appel f2
3. main appel f1

Schématisez l'état de la pile pendant l'exécutions de f2.

Répondez aux mêmes questions que pour le programme précédent.

Bonus ! Écrivez un programme récursif qui affiche n frame (l'adresse d'une variable contenue dans la frame de la fonction)

# 2 allocation de mémoire




Des requêtes d'allocation de mémoire arrivent dans cet ordre: 20 Ko, 10 Ko, 5 Ko et 25 Ko.

**Question 1** A quelles adresses sont alloués les blocs si on utilise la politique **First Fit** ?

**Question 2** A quelles adresses sont alloués les blocs si on utilise la politique **Best Fit** ?

**Question 3** A quelles adresses sont alloués les blocs si on utilise la politique **Worst Fit** ?

## 2.1 Pointeurs

```c
#include <stdio.h>
#include <stdlib.h>

typedef void* address_t;

int main() {
  address_t *x;

  x = malloc(sizeof(address_t));
  *x = (void*)3735928559;

  printf(" x = %p\n", (address_t)x);
  printf("*x = %p\n", (address_t)*x);
  printf("&x = %p\n", (address_t)&x);
}
```

Expliquez la sortie du programme précédent.

# 3 Mémoire virtuelle, pagination

Pour simplifier les calculs, nous allons utiliser des pages de taille 1000 octets. Les cadres ont donc une taille de 1000 octets également. Normalement, la taille d'une page serait une puissance de 2 : 4096 octets en général.

La table de pages est la suivante :

| Page | In/Out | Cadre |
|---|---|---|
| 0 | In | 20 |
| 1 | Out | 22 |
| 2 | In | 37 |
| 3 | In | 15 |
| 4 | Out | 97 |
| 5 | Out | 15 |
| 6 | In | 03 |
| 7 | In | 11 |

Parmi les adresses virtuelles suivantes, lesquelles génèrent un défaut de page ?

- 6451
- 5421
- 123
- 3156

Pour celles qui ne génèrent pas de défaut de page, quelle est l'adresse physique référencé ?

## 3.1 Algorithme

Un système possède 3 cadres de page et fait référence aux pages suivantes :
0, 9, 0, 1, 8, 1, 8, 7, 8, 7, 1, 2, 8, 2, 7, 8, 2, 3, 8, 3

Combien de défauts de page sont générés si on utilise :

- le remplacement FIFO
- le remplacement LRU

## 3.2 Adressage

Supposons qu'un ordinateur utilise des adresses 32 bits et que la table des pages soit une table à 2 niveaux. Les adresses virtuelles sont découpées en 3 parties et la taille des pages est de 4ko:

- une partie sur a bits pour le premier niveau de table,
- une partie sur b bits pour le deuxième niveau de table,
- une partie sur o bits pour l'offset dans la page.

On a a + b + o = 32.
Quelles relations y-a-t il entre la taille d'une page et a, b et o ?
Même question pour le nombre de pages disponibles.